

Neighbourhood WatchDog

Technical Deployment Document

| We don't just detect threats, we predict them.

Team Intrepid · COS 301 Capstone Project · University of Pretoria

In partnership with EPI-USE Africa

Version 1.0 · 2026

Table of Contents

1. Overview
2. Architecture Overview
3. Prerequisites
4. Infrastructure - AWS Setup
5. Repository and Environment Setup
6. Services Reference
7. Deployment Steps
8. Database Migrations
9. CI/CD Pipeline
10. WatchDog Agent Deployment
11. Health Checks and Verification
12. Monitoring
13. Rollback Procedure
14. Troubleshooting

1. Overview

This document provides a complete technical reference for deploying, configuring, and maintaining the Neighbourhood WatchDog platform in a production environment.

The system is deployed on AWS in the Europe (Stockholm) region and is accessible at neighbourhoodwatchdog.co.za. All services are containerised with Docker and orchestrated via Docker Compose. SSL termination and reverse proxying are handled by Caddy.

2. Architecture Overview

Services include Backend (FastAPI), Frontend (Next.js), AI (YOLOv8), MediaMTX, PostgreSQL/PostGIS and Caddy.

Traffic flow:

- HTTP/HTTPS traffic enters through Caddy (80/443).
- API requests are proxied to the backend (8000).
- Dashboard requests are proxied to the frontend (3000).
- WebRTC streams are served by MediaMTX (8889).
- RTSP streams arrive on 8554.

3. Prerequisites

Deployment Machine

- Git
- SSH access to EC2
- AWS CLI

EC2 Server

- Ubuntu 22.04+
- Docker Engine
- Docker Compose v2
- Git

Install Docker

```
ssh -i your-key.pem ubuntu@your-ec2-ip
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker ubuntu
newgrp docker
docker --version
docker compose version
```

4. Infrastructure - AWS Setup

EC2 Instance

The system runs in `af-south-1` (Cape Town) .

Security Group Rules

Open ports: 22, 80, 443, 8554, 8889, 8189 and internal ports 9997 and 5432.

AWS Cognito

- Create a User Pool in `af-south-1` .
- Enable MFA (TOTP).
- Configure App Client.
- Create the required user groups.

Domain and SSL

```
neighbourhoodwatchdog.co.za {  
  reverse_proxy /api/* backend:8000  
  reverse_proxy /ws/* backend:8000  
  reverse_proxy /* frontend:3000  
}
```

```
stream.neighbourhoodwatchdog.co.za {  
  reverse_proxy /* mediamtx:8889  
}
```

5. Repository and Environment Setup

Clone Repository

```
git clone https://github.com/COS301-SE-2026/Neighbourhood-WatchDog.git  
cd Neighbourhood-WatchDog
```

Environment Variables

Important variables include:

- `POSTGRES_USER`
- `POSTGRES_PASSWORD`

- POSTGRES_DB
- SECRET_KEY
- AWS_REGION
- COGNITO_USER_POOL_ID
- COGNITO_CLIENT_ID
- NEXT_PUBLIC_API_URL
- NEXT_PUBLIC_MEDIAMTX_WEBRTC_URL
- MTX_WEBRTCADDITIONALHOSTS
- BACKEND_URL

6. Services Reference

Service	Internal Address	Ports
backend	backend:8000	8000
frontend	frontend:3000	3000
ai	ai:8001	8001
mediamtx	mediamtx:8554	8554,8889,8189,9997
postgres	postgres:5432	5432
caddy	caddy:80	80,443

7. Deployment Steps

First-time Deployment

```
git clone https://github.com/COS301-SE-2026/Neighbourhood-WatchDog.git
cd Neighbourhood-WatchDog
cp .env.example .env
docker compose build
docker compose up -d postgres
docker compose run --rm backend alembic upgrade head
docker compose up -d
```

Updating a Deployment

```
git pull origin main
docker compose build
docker compose run --rm backend alembic upgrade head
docker compose up -d
```

8. Database Migrations

```
docker compose run --rm backend alembic upgrade head
docker compose run --rm backend alembic current
docker compose run --rm backend alembic downgrade -1
```

9. CI/CD Pipeline

The GitHub Actions pipeline:

1. Runs backend tests.
2. Runs frontend tests.
3. Runs Playwright tests.
4. Deploys to EC2 after successful validation.

10. WatchDog Agent Deployment

The agent:

- Pulls RTSP streams locally.
- Sends frames to the AI service.
- Relays streams to MediaMTX.
- Posts detection events to the backend.

11. Health Checks

```
docker compose ps
curl https://neighbourhoodwatchdog.co.za/api/health
docker compose exec postgres pg_isready -U $POSTGRES_USER
```

12. Monitoring

- Configure restart policies.
- Monitor disk usage using `df -h`.
- Perform regular PostgreSQL backups.

13. Rollback Procedure

```
git checkout <commit-hash>
docker compose build
docker compose up -d
```

14. Troubleshooting

Common issues include:

- Containers restarting repeatedly.
- SSL certificate provisioning failures.
- WebRTC streaming problems.
- Backend 500 errors.
- RTSP connectivity issues.
- Notification delivery failures.